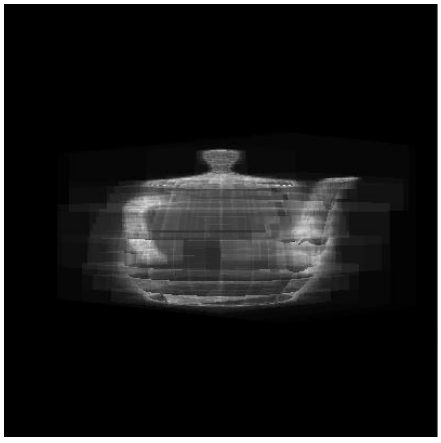
Evaluating BVH subdivision heuristics

Márton Mohácsi - 16211016

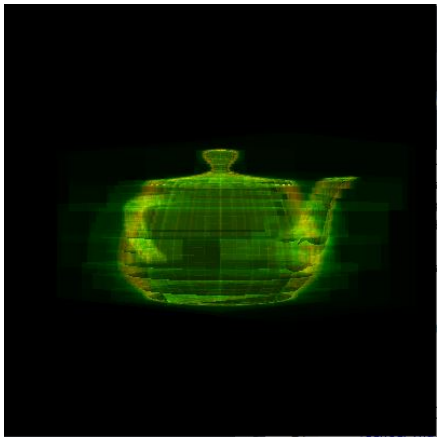
January 22, 2026

(a) Scene as is

(b) Intersections



(c) Traversals



(d) Combined intersections and traversals

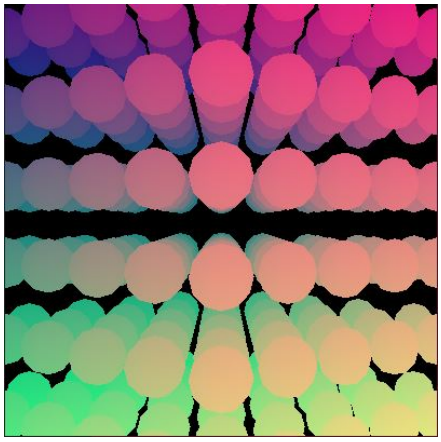
Figure 1: The four textures the project renders

1 Project description and build instructions

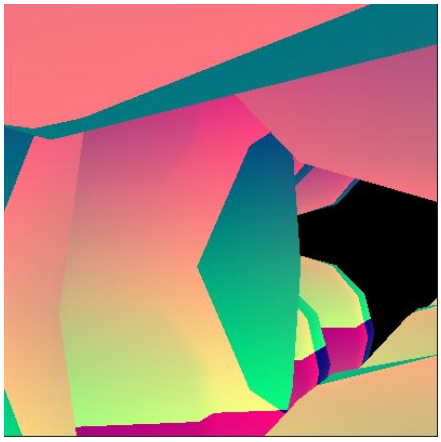
1.1 Project

This project compares between multiple BVH subdivision heuristics. It implements a very basic ray tracing, with no bounces beyond the first. For any given render, the project creates 4 textures that can be selected between:

- The first is the scene rendered as is, with the models colored based on the coordinates of the point where the given ray from the camera hits them. This can be seen in Figure 1a.
- The second is a grayscale render, where every pixel is colored in accordance with how many intersection tests (with triangles) the ray cast through that pixel had to do to complete the render. The blackest pixel corresponds to the minimum amount of intersections, and the whitest pixel corresponds to the maximum amount of intersections. This can be seen in Figure 1b.
- The third is a grayscale render where every pixel is colored in accordance with how many traversal steps through the BVH structure the ray cast through that pixel had to do to complete the render. The blackest pixel corresponds to the minimum amount of traversal steps, and the whitest pixel corresponds to the maximum amount of traversal steps. This can be seen in Figure 1c.
- The fourth is a colored image combining the previously mentioned two measures, the intersection measures are shown in the red channel, and the traversal steps are shown in the green channel of any given pixel. This can be seen in Figure 1d.



(a) LOA scene



(b) HOA scene

Figure 2: The high and low occlusion scenes of the project

To compare the different BVH subdivision heuristics 3 different scenes are used:

- The first scene is called **base**. It contains a single Utah teapot roughly centered on the origin. It can be seen in Figure 1a. The scene has roughly 6 thousand primitives.
- The second scene is called **low occlusion A** or **LOA**. It contains 216 spheres with 60 faces each arranged in a 6x6x6 cube centered on the origin. It can be seen in Figure 2a. The scene has roughly 13 thousand primitives.
- The third scene is called **high occlusion A** or **HOA**. It contains 216 spheres with 60 faces each arranged in a 6x6x6 cube centered on the origin. These spheres are bigger and more spread out than in LOA. Additionally this scene also contains a sphere chopped in half, which is big enough to always obstruct a large percentage of the view. The scene can be seen in Figure2b. The HOA scene has a bug that makes its colors incorrect. On the left we can see the half sphere, and on the right we can see the full spheres in the distance, and some very close to the camera. The scene has roughly 13 thousand primitives.

Finally, the project implements five different BVH subdivision heuristics referenced in (Meister et al., 2021). These heuristics are:

- naive
- SAH
- Ray Distribution Heuristic or RDH
- RDH blended with SAH
- Occlusion Heuristic or OH

These heuristics are detailed below.

1.2 Build instructions

The project can be built using Visual Studio 2026 Community edition, it uses GLFW, so it expects the GLFW_PATH macro to be defined.

2 Methods for evaluating the heuristics

This project is far from optimized in many aspects. To make things as fair as possible for our evaluations we will compare metrics instead of time spent when evaluating the efficiency of any given heuristic. This applies to both construction and rendering. The subsections below will detail these metrics.

2.1 Construction

For all heuristics except for the naive heuristic we will evaluate cost functions when attempting to split a BVH node. When splitting this way we want to find the splitting plane that minimizes $C_1 + C_2$ where C_1 and C_2 is the cost of the newly created BVH nodes.

To do this, we need to evaluate every possible split. This means we have to put the splitting plane between every two neighboring primitives by each axis and evaluate the cost of the BVH nodes this plane would create.

This evaluation will always involve the creation of the hypothetical BVH nodes: determining the extent of their bounding boxes and determining the number of primitives inside the bounding box. After we created our new hypothetical BVH nodes we need to determine the summarized cost of them.

Let us call the time spent constructing and evaluating a pair of hypothetical BVH nodes C. Although C will take more or less time depending on the number of primitives inside the BVH node we are splitting it will still give us a rough estimate of how much time is taken up by just the creation of hypothetical bounding boxes and evaulating the cost function associated with them.

Some of our heuristics will use bounding box intersections with rays. Let us call the time spent intersecting a bounding box A. Some of our heuristics will use primitive intersection with rays. Let us call the time spent intersecting a primitive P. P will always denote the same amount of time since the only primitive is a triangle.

2.2 Rendering

When rendering is evaluated we will use two metrics. The first one called traversal, this denotes the number of traversal steps through the BVH a given ray had to take to create the render. The second one is called intersection, this denotes the number of primitive intersections a ray had to do to create the render.

The rendering performance of the BVH subdivision methods will be evaluated by taking measurements of how many intersection and traversal steps were needed to render a given scene. These measurements will be taken by spinning the camera around the origo of the scene at a resolution of one degree per spin. Meaning each scene will be rendered 360 times.

For each render we measure the average and maximum intersections and traversals taken by each pixel to compute the render. For each we note the average and maximum value for each render. In summary we will have 8 values for each BVH for each scene:

- **AoAI**: average of average intersections. This tells us that on average how many intersections it takes to render this scene. We just need to multiply this value with the resolution of our screen.
- **AoMI**: average of maximum intersections. This tells us that on average how long we spend rendering the pixel that requires the most amount of

intersections for this scene per angle. Investigating outliers can help us understand why our BVH might be slow in some cases.

- **MoAI**: maximum of average intersections. By multiplying this value with the screen resolution we get a strict upper bound for how many intersection tests we need to render our scene from any angle. If this value is close to AoAI that means all the intersections for each angle of the scene can be calculated in a similar amount of time.
- **MoMI**: maximum of maximum intersections. This value tells us how many intersections were needed to render the pixel that required the most intersections for any angle of this scene. If this value is much bigger than AoMI that means that at least for one (or more) angles of this scene our BVH is not fast, but overall it is. It can help us understand which scenarios our BVH performs badly in.
- **AoAT**: average of average traversals. This tells us that on average how many traversal steps it takes to render this scene. We just need to multiply this value with the resolution of our screen.
- **AoMT**: average of maximum traversals. This tells us that on average how long we spend rendering the pixel that requires the most amount of traversal steps for this scene per angle. Investigating outliers can help us understand why our BVH might be slow in some cases.
- **MoAT**: maximum of average traversals. By multiplying this value with the screen resolution we get a strict upper bound for how many traversal steps we need to render our scene from any angle. If this value is close to AoAT that means all the intersections for each angle of the scene can be calculated in a similar amount of time.
- **MoMT**: maximum of maximum traversals. This value tells us how many traversal steps were needed to render the pixel that required the most traversal steps for any angle of this scene. If this value is much bigger than AoMT that means that at least for one (or more) angles of this scene our BVH is not fast, but overall it is. It can help us understand which scenarios our BVH performs badly in.

3 SAH and naive subdivision

The naive and SAH subdivision methods for BVHs will be taken as a baseline for rendering performance. Here naive subdivision method refers to always splitting the bounding boxes in half along their longest axis.

The SAH is a good baseline because it is widely used for its relatively quick construction time and good performance during rendering. If any subdivision heuristic can perform better than it during rendering, then its worth considering for rendering applications.

The naive subdivision has the quickest construction out of all evaluated subdivision heuristics by far. On the other hand, it does not perform great during rendering. If any subdivision heuristic performs worse during than the naive subdivision it should not be considered. Therefore the naive subdivision provides a good sanity check for us.

3.1 Construction

The construction with the naive subdivision is extremely quick, for our purposes we will treat it as if it took no time, so a construction time of $C*0+A*0+P*0 \simeq 0$.

When constructing a BVH using SAH, we don't split a node further if no splitting plane exists where $C_1 + C_2 < C_P$, where C_P denotes the cost of the parent.

Metrics	base SAH	LOA SAH	HOA SAH
C	0.25	0.56	0.56
A	0	0	0
P	0	0	0

Table 1: Construction metrics of SAH BVH per scene expressed in millions

During the construction of a BVH using SAH we do not need to intersect with a bounding box or a primitive so the construction time of the BVH can be expressed using only C. The time spent constructing the BVH expressed with C by the SAH algorithm can be seen in Table 1.

The SAH construction algorithm will depend on the geometry we want to construct the BVH for. At every level of the BVH each primitive will be present in the nodes of the tree (during construction) exactly once. For each primitive there will be one possible splitting plane for splitting up the node. Since C is equal to testing a splitting plane the amount of C in a given construction of an SAH BVH will heavily correlate with the number of primitives in the scene. This can be seen demonstrated in our results.

The amount of C will not correlate strictly with the number of primitives in a scene, because the resulting BVH will not necessarily be a balanced tree. Some leaves might have more primitives in them than others.

3.2 Rendering

We can look at the AoAI and AoAT values as the "speed" of our render. The AoAT values are roughly equal for both navie and SAH methods, this is due to the fact that the depth of these trees are roughly equal. However the AoAI is much lower for the SAH method, this is why the cost function exists, to minimize the number of primitives we have to look through in the leaves of the BVH tree.

Metrics	base naive	base SAH	LOA naive	LOA SAH	HOA naive	HOA SAH
AoAI	2.04	0.66	18.33	6.82	23.38	9.46
AoMI	147.73	27.75	76.39	30.44	59.62	27.24
MoAI	3.27	1.04	32.32	11.83	45.91	18.42
MoMI	343	45	131	45	95	41
AoAT	12.04	10.09	73.79	69.41	99.18	88.52
AoMT	165.03	159.97	200.73	190.69	188.82	174.81
MoAT	17.15	14.02	111.43	106.23	172.98	147.70
MoMT	267	247	265	245	273	237

Table 2: Rendering metrics for naive and SAH BVH per scene

This pattern holds for our other metrics as well. The traversal steps are roughly equal in with both methods, however there are drastically less intersection tests for the SAH method. Furthermore the AoMI values are much closer, even relatively to the MoMI values for the SAH method, meaning the amount of intersections per pixels are more equally distributed.

4 Ray Distribution Heuristic

The Ray Distribution Heuristic (or RDH) (Bittner & Havran, 2009) is a subdivision method that, similarly to the SAH tries to estimate how many rays will hit the two bounding boxes created by any given splitting plane.

The RDH uses a so called ray distribution, which is a list of rays, to compare bounding boxes against. Meaning, that for each evaluated bounding box its cost will be NR, where N is the number of primitives inside the bounding box, and R is the number of rays from the ray distribution that intersect the bounding box.

This method was created for splitting up kd-trees. In the case of kd-trees, the cost function described above was not sufficient to creating an efficient split. For this reason, the authors proposed a cost function that blends both the RDH and the SAH approach.

The blending approach proposes that the cost of any given split should be determined as such:

$$wC_{RDH} + (1 - w)C_{SAH}$$

Where C_{RDH} is the RDH cost and C_{SAH} is the SAH cost. Furthermore the authors proposed that the blending weight w should be determined by the following equation:

$$w = \alpha(1 - \frac{1}{1 + \beta|R|})$$

Where $\alpha = 0.9$, $\beta = 0.1$ and $|R|$ is the number of rays intersecting the parent node.

Similarly to SAH, both RDH and the blended method only split a BVH node if there is a splitting plane that creates two new BVH nodes such that the sum of their costs is smaller than the cost of the BVH node we are splitting.

The ray distributions are created with some number of viewpoints and an NxN "pattern". For each viewpoint, a ray is determined for each non-overlapping NxN patch of the "screen". Meaning that for a rendering size of 400x400 and a pattern size of $N = 20$, there will be $\frac{400}{20}\frac{400}{20} = 20 * 20 = 400$ rays determined for each viewpoint.

4.1 Construction

The construction of BVH using RDH or the blended method will start with creating the ray distributions. When calling to create one of these BVHs, the number of viewpoints and the pattern size can be given as parameters. The viewpoints will be equi-distant on the boundary of a circle on the y plane, always facing the origin.

After we created our ray distribution we start splitting the BVH nodes. When trying to split a BVH node we have to try intersecting each possible hypothetical bounding box with each ray in our ray distribution. This will cause us to do a lot of intersection checks. To help minimize this cost the project implements a helper structure that denotes which rays intersected the bounding box of the parent, so that we don't have to check for rays from the distribution that already missed the bounding box of the parent. This helper structure is newly calculated each time we attempt to split a BVH node. This can be done economically while calculating the cost of the parent node.

A further optimization for the construction would be passing this helper structure along while subdividing the nodes. This way, we would not need to recalculate the rays intersecting the parent each time we attempt to split a node. However, this optimization was not implemented.

Similarly to SAH both RDH and the blended method only split a BVH node if a splitting plane exists where $C_1 + C_2 < C_P$.

	Metrics	RDH_6_4	RDH_6_10	RDH_10_12	RDH_20_20
base	C	0.25	0.21	0.22	0.19
	A	1718.59	257.37	287.85	188.76
	P	0	0	0	0
LOA	C	0.56	0.56	0.55	0.56
	A	2468.55	1766.68	2117.91	1469.79
	P	0	0	0	0
HOA	C	0.33	0.33	0.35	0.34
	A	151.74	2071.26	2509.4	1724.05
	P	0	0	0	0

Table 3: Construction metrics of RDH BVH per scene expressed in millions

For each scene four BVHs were constructed using the RDH method with

different parameters. They are all called RDH_x_y where x is the number of viewpoints they have and y is the size of the pattern. The constructed BVHs are: RDH_6_4, RDH_6_10, RDH_10_12 and RDH_20_20. The construction metrics for each of these BVHs can be seen in Table 3.

The amount of rays in the ray distribution for any given RDH_x_y can be calculated using the following equation: $\#R = x * \lfloor \frac{400}{y} \rfloor^2$. Thus we can calculate that RDH_6_4 has 60000 rays, RDH_6_10 has 9600 rays, RDH_10_12 has 10890 rays and RDH_20_20 has 8000 rays in their respective ray distributions. The amount of A in their construction times will correlate with the number of rays in their ray distributions, as we can verify above.

The reason the difference between A values is less stark in the LOA and HOA scenes is that the primitives are more spread out for each viewpoint, while for the base scene all the primitives are concentrated to the middle of the scene. Since RDH_6_4 has denser rays in its ray distribution more rays from it will hit the primitives clustered in the middle of the scene. The optimization that disregards all rays that don't hit the bounding box of the node we are attempting to split needs to keep more rays in consideration for a ray distribution that is denser.

The reasoning for the C amounts described for the SAH method applies here as well. The difference is that in the HOA scene there are several primitives that are always off screen. Therefore no rays from any ray distributions will hit them, and they will not be able to split into different BVH nodes, since the cost is the function of only the number of rays hitting bounding boxes.

	Metrics	blended_6_2	blended_6_4	blended_6_10	blended_10_12	blended_20_20
base	C	0.25	0.25	0.25	0.25	0.25
	A	2667.39	1740.79	280.94	317.39	218.37
	P	0	0	0	0	0
LOA	C	0.56	0.56	0.56	0.56	0.56
	A	1314.86	2468.55	1767.22	2120.84	1472.15
	P	0	0	0	0	0
HOA	C	0.58	0.57	0.57	0.57	0.56
	A	2587.73	648.5	2164.57	2608.1	1803.02
	P	0	0	0	0	0

Table 4: Construction metrics of blended BVH per scene

For each scene five BVHs were constructed with the blended method with different parameters. They are all called blended_x_y where x is the number of viewpoints they have and y is the size of the pattern. The constructed BVHs are: blended_6_2, blended_6_4, blended_6_10, blended_10_12 and blended_20_20. The construction metrics for each of these BVHs can be seen in Table 4. The reasoning for the construction results of the BVHs with the RDH method applies here as well with a few small differences.

Since we can always split up a node, even if it only has primitives that are not hit by any ray from the ray distribution, the depth of the tree will be similar

to the depth of the tree using the SAH method. As we have discussed previously the C amount correlates to the depth of the tree, since each new full level of the tree adds a C for each primitive in the scene. We can see this reflected in the C amounts above.

Compared to the RDH method the blended method has slightly more cost evaluation and bounding box intersection operations for scenes where most primitives are visible from most viewpoints. However for scenes where most primitives are not visible for most viewpoints the blended method will have a significantly longer construction. However it will also likely create a significantly better BVH.

For both the RDH and blended method, the BVH that was created using 6 viewpoints and a pattern size of 4x4 uses significantly less bounding box traversals as the BVHs with the same amount of viewpoints but bigger, or smaller pattern sizes. Since as we discussed the C_{RDH} is sensitive to the rays in the ray distribution, the combination of the scene and parameters for the BVH construction must result in a ray distribution that hits few candidate bounding boxes.

4.2 Rendering

We can see the rendering metrics for each scene for the BVHs constructed using the RDH method in Table 5.

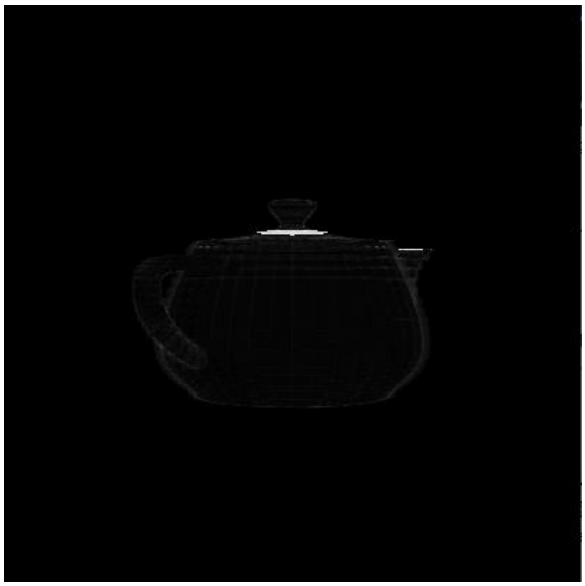


Figure 3: High intersection counts for renders using RDH_6_4

For the base scene, based on the "speed" metric of AoAI + AoAT, the RDH_6_4 BVH seems to perform the best. This is due to the fact that there is

	Metrics	RDH_6_4	RDH_6_10	RDH_10_12	RDH_20_20
base	AoAI	0.72	6.09	6.74	18.18
	AoMI	102.12	798.33	918.45	1028.32
	MoAI	1.03	6.77	7.71	22.26
	MoMI	129	849	1026	1291
	AoAT	8.42	8.17	7.68	7.10
	AoMT	138.48	121.73	119.89	106.29
	MoAT	10.37	10.00	9.38	8.64
	MoMT	221	163	177	145
LOA	AoAI	6.50	6.59	10.06	6.87
	AoMI	30.79	31.11	240.07	56.04
	MoAI	10.12	10.25	13.63	10.76
	MoMI	45	45	254	77
	AoAT	55.01	54.58	54.44	54.79
	AoMT	155.57	153.42	154.76	153.68
	MoAT	70.73	69.95	70.40	71.56
	MoMT	193	187	191	189
HOA	AoAI	10.56	10.82	9.38	11.65
	AoMI	1204.43	1219.64	114.95	1347.87
	MoAI	19.96	20.38	15.96	21.57
	MoMI	3314	3298	464	3563
	AoAT	76.67	73.18	73.51	72.86
	AoMT	177.75	171.69	165.27	164.32
	MoAT	102.98	97.54	100.96	100.02
	MoMT	219	217	211	205

Table 5: Rendering metrics for RDH BVH per scene

an area of the geometry that is missed by all rays from all viewpoints. Figure 3 depicts the intersection view of the base scene using RDH_6_4. Even with the BVH that has the densest ray distribution there is an area of the render that needs to do a lot of intersections, presumably because a lot of triangles are in the same bounding box.

This problem only gets worse for BVHs with less dense ray distributions. Conversely the MoAT and MoMT values decrease as the ray distributions decrease. This is also because there will be a bunch of triangles not hit by any ray, therefore included into bounding boxes missed by all the rays of the ray distributions.

For the LOA scene based on the "speed" metric the RDH_6_10 BVH seems to perform the best, but it is only slightly better than the RDH_6_4 BVH. In this scene most triangles are hit by most rays in the ray distributions, this is due to the fact that the triangles are larger than in the base scene. We can see this in the data as well by seeing that the metrics concerning maximum amount of intersections is low. Except for RDH_10_12, here the same problem comes up as with the base scene, due to unfortunate placement of viewpoints this BVH

contains a bounding box containing many triangles.

For the HOA scene based on the "speed" metric the RDH_10_12 BVH seems to perform the best. Again we see the large MoMI values indicating the presence of bounding boxes containing many primitives not hit by rays from the ray distributions. Besides this, similarly to the LOA scene the traversal metrics are all roughly the same for each BVH.

As a conclusion about rendering using BVHs constructed with the RDH and blended methods, we need to be careful to place viewpoints in a way that most, or preferably all primitives are hit or at least nearly missed by a ray in the ray distribution. These measures were also taken with the same y coordinate as the viewpoints for the ray distributions, and also facing the same direction, so this is a fairly ideal scenario for this method.

	Metrics	blended_6_2	blended_6_4	blended_6_10	blended_10_12	blended_20_20
base	AoAI	0.64	0.63	0.63	0.63	0.64
	AoMI	30.54	30.11	29.64	29.82	30.07
	MoAI	0.92	0.93	0.91	0.93	0.95
	MoMI	45	45	45	45	45
	AoAT	8.49	8.54	8.78	8.64	9.17
	AoMT	144.23	147.03	157.72	159.19	159.94
	MoAT	10.46	10.54	10.84	10.77	11.51
	MoMT	225	235	241	243	231
LOA	AoAI	6.53	6.50	6.50	6.50	6.57
	AoMI	30.73	30.79	30.76	30.75	30.78
	MoAI	10.21	10.12	10.15	10.13	10.35
	MoMI	45	45	45	45	45
	AoAT	55.29	55.10	55.27	55.23	55.83
	AoMT	155.26	155.79	156.16	156.16	157.32
	MoAT	71.18	70.88	71.13	71.77	73.58
	MoMT	189	193	195	195	191
HOA	AoAI	9.52	9.52	9.39	9.36	9.35
	AoMI	29.27	29.31	28.79	28.95	28.87
	MoAI	15.83	15.82	15.84	15.87	15.82
	MoMI	41	41	41	41	41
	AoAT	77.10	77.09	75.07	73.84	73.77
	AoMT	182.24	182.52	173.11	168.63	172.27
	MoAT	104.55	104.40	102.47	101.78	101.65
	MoMT	241	239	225	213	215

Table 6: Rendering metrics for blended BVH per scene

We can see the rendering metrics for each scene for the BVHs constructed using the blended method in Table 6. We can immediately notice that regardless of ray count in the ray distributions, each BVH produces roughly equal metrics for rendering for all metrics.

Based on the "speed" metric for both the base and the LOA scene the

blended_6_4 BVH performs the best. For the HOA scene the blended_20_20 has the best "speed" value.

As we can observe from the metrics related to maximum intersections there are no longer bounding boxes missed by all rays in ray distributions containing a large amount of primitives we have to check. The blending of the RDH and SAH costs prevents this from happening.

Intuition would dictate that the BVH with the densest ray distribution would provide the best performance. However the sensitivity to the placement of the viewpoints that was present for the BVHs created with only the RDH method is also present here. This sensitivity however is smoothed out by the blended cost function.

When comparing BVHs constructed with the RDH method against BVHs constructed with the blended method the blended BVHs vastly outperform the RDH metod in the metrics related to intersections, and the RDH BVHs slightly outperform the blended method in the metrics related to traversals. The "speed" of the blended BVHs is generally better, this is backed up by intuition since the blended BVHs are less sensitive to the potential unfortunate placements of the viewpoints.

We can also observe from the comparison between the AoAI and MoAI, and the comparison between AoAT and MoAT metrics that for the blended method these values are closer. This means that the slowest render of the blended method will take less time than the slowest method of the RDH method. At the same time their "speed" values are similar, which means that the blended method will produce a render in more regular intervals, which is desirable for real time applications.

5 Occlusion Heuristic

The Occlusion Heuristic (or OH) (Vinkler, Havran, & Sochor, 2012) is a subdivision method that tries to divide any given BVH node into a node containing mostly visible primitives and a node containing mostly non-visible primitives.

This heuristic was proposed for two main use cases. First animations that have a high degree of frame to frame coherence such that we can estimate visible ptimitives with a high degree of accuracy. Second rendering use cases where we can rebuild the BVH on the fly, and we collect visibility metrics while rendering. Even in these two cases we only get an estimation of which primitives are visible for any given frame and which primitives are non-visible.

The heuristic tries to split primitives in a given node into two nodes, while trying to make it so that the first has mostly visible primitives and the second has mostly non-visible primitives.

The cost function of a given hypothetical BVH node looks like the following:

$$p_1 = w\frac{N_1^V}{N_1^V + N_2^V} + (1 - w)\frac{C_{SAH1}}{C_{SAH}}$$

where $w = 0.9$, N_1^V is the number of visible primitives in the first bounding

box, thus $N_1^V + N_2^V$ will be the number of visible primitives in the BVH node we are splitting, C_{SAH1} is the SAH cost for the first bounding box, and C_{SAH} is the SAH cost for the parent node.

The total cost of a given split will be: $p_1 N_1 + p_2 N_2$, where N_1 and N_2 are the number of primitives in the first and second bounding boxes.

The paper argues that this way of splitting BVH nodes is only meaningful close to the root of the tree. After a given depth each BVH node will have almost only visible or almost only non-visible primitives. To this end this cost function is only used at depth which is less than $\frac{1}{2}log(N)$ where N is the number of primitives in the scene. After this depth a simple SAH cost is used to finish the subdivision of the BVH.

5.1 Construction

Since we want to compare between BVH construction methods that are not on-the-fly, the implementation of this heuristic was modified for our purposes.

We will specify some number of viewpoints, "render" the scene from each of them, and note for each primitive how many viewpoints it was visible from. We will call this value the visibility measure of the primitive. After this we will construct our BVH, with the cost function modified as such:

$$p_1 = w\frac{N_1^{\#V}}{N_1^{\#V} + N_2^{\#V}} + (1-w)\frac{C_{SAH1}}{C_{SAH}}$$

where $N^{\#V}$ denotes the sum of the visibility measures for each primitive in the given bounding box.

As the cost function of the parent node before a depth of $\frac{1}{2}log(N)$ we will simply use the number of primitives inside it. After this depth the cost function of the parent will be the same cost function as with the SAH split.

Similarly to SAH this method will only split a BVH node if a splitting plane exists where $C_1 + C_2 < C_P$.

To construct our OH BVH we need to give the number of viewpoints as parameters. Similarly to the RDH ray distributions the viewpoints will be equidistant on the boundary of a circle on the y plane, always facing the origin.

Before we can determine the visibility measures for our primitives, we first construct a simple SAH BVH that will help us speed up the rendering of the scene from our viewpoints. The construction metrics of this BVH will be added to the construction metrics of the OH BVH we are constructing. With the use of our helper BVH we will determine the visibility measures for our primitives.

After this the construction of the OH BVH will not take much more time than the construction of a simple SAH BVH. The extra costs are a few more operations per C to determine the cost of the given split.

For each scene three BVHs were constructed with the OH method with different parameters. They are all called OH_x where x is the number of viewpoints they have. The constructed BVHs are: OH_6, OH_10 and OH_20. The construction metrics for each of these BVHs can be seen in Table 7.

	Metrics	OH_6	OH_10	OH_20	SAH
base	C	0.5	0.5	0.5	0.25
	A	12.47	20.86	41.67	0
	P	0.95	1.58	3.15	0
LOA	C	1.12	1.12	1.12	0.56
	A	94.82	158.09	316.15	0
	P	11.2	18.77	37.55	0
HOA	C	0.99	1.15	1.15	0.56
	A	124.38	207.29	414.49	0
	P	15.72	26.68	53.58	0

Table 7: Construction metrics of OH BVH per scene expressed in million

For each BVH we construct with the OH method we have to construct a BVH with the SAH method. The depth of the BVH using the OH method will be very similar to the depth of the BVH using the SAH method. This is because after a depth of $\frac{1}{2}log(N)$ the cost function of the BVH using the OH method will be the same as the cost function of the BVH using the SAH method. So if a branch of the tree reaches below this depth it will likely be split to a similar degree with the OH method, as it would be with the SAH method. This can be seen reflected in the C amounts above.

The A and P amounts are purely from traversing the helper BVH. They are correlating with the number of viewpoints we need to "render" for creating the visibility metrics for the primitives. We can see this bear out on the A and P values above, both of these values are almost exactly double for OH_20 as they are for OH_10.

In the case of the OH_6 BVH for the HOA scene the reduced C metric indicates that the splitting of the BVH nodes terminated before the depth where the SAH cost would take over as the cost function. This scene has a lot of primitives close to the potential viewpoints. This means that if there are too few viewpoints, a lot of primitives will have a visibility metric of 0. When trying to split a node with with close to zero triangles with non-zero visibility metrics the cost of the split might always be lower than the cost of the parent, namely the number of primitives in the parent node. When this is the case no split is made, and the BVH is left with a node that has many primitives inside its bounding box, and a shallower tree than othervise.

5.2 Rendering

We can see the rendering metrics for each scene for the BVHs constructed using the OH method in Table 8. For all cases except for one the OH method produces essentially the same values for the same scenes. Based on the "speed" metric the best BVH using the OH method for the base and LOA scenes is OH_6. The best BVH for the HOA scene is the OH_20.

For the HOA scene the BVHs don't produce similar results. Intuition dic-

	Metrics	OH_6	OH_10	OH_20
base	AoAI	0.65	0.65	0.65
	AoMI	27.76	27.51	27.51
	MoAI	1.03	1.03	1.03
	MoMI	45	45	45
	AoAT	9.94	9.96	9.95
	AoMT	164.56	162.76	162.84
	MoAT	13.65	13.69	13.68
	MoMT	241	249	249
LOA	AoAI	6.80	6.80	6.80
	AoMI	30.48	30.48	30.48
	MoAI	11.55	11.55	11.55
	MoMI	45	45	45
	AoAT	70.37	70.37	70.37
	AoMT	192.23	192.23	192.23
	MoAT	106.00	106.00	106.00
	MoMT	253	253	253
HOA	AoAI	1747.46	9.58	9.49
	AoMI	2807.73	26.12	26.10
	MoAI	4553.20	18.49	18.46
	MoMI	4574	41	41
	AoAT	101.93	114.03	106.83
	AoMT	195.06	206.61	200.31
	MoAT	161.16	199.47	192.66
	MoMT	271	311	309

Table 8: Rendering metrics for OH BVH per scene

tates that the performance of this BVH should go up as the number of viewpoints go up that were used during their constructions. The less viewpoints we have the less accurately our visibility measures estimate which triangles are the most likely to be visible for any angle.

In the case of the HOA scene and OH_6, as discussed in the construction chapter, this BVH is going to be too shallow with nodes that contain many primitives. The cause of this is a high occlusion scene and too few viewpoints for creating the BVH. We can see that for this scene 10 viewpoints are enough to create a performant BVH.

We have to acknowledge that the evaulation environment of these BVHs is ideal. The angles from which we measured our rendering metrics are all potential input viewpoints for our BVH. Furthermore each viewpoint used to create the BVHs will be used to measure from as well.

Metrics	naive	SAH	RDH_6_4	blended_6_4	OH_6
AoAI	2.04	0.66	0.72	0.63	0.65
AoMI	147.73	27.75	102.12	30.11	27.76
MoAI	3.27	1.04	1.03	0.93	1.03
MoMI	343	45	129	45	45
AoAT	12.04	10.09	8.42	8.54	9.94
AoMT	165.03	159.97	138.48	147.03	164.56
MoAT	17.15	14.02	10.37	10.54	13.65
MoMT	267	247	221	235	241
C	0	0.25	0.25	0.25	0.5
A	0	0	1718.59	1740.79	12.47
P	0	0	0	0	0.95

Table 9: Rendering and construction (in millions) metrics for base scene

6 Comparison of evaulated Heuristics

In Table 9 we see the rendering and construction metrics for the BVH with the best "speed" metric from each category for the base scene. As a first sanity check we can see that the naive subdivision method produces the slowest BVH for all metrics.

The SAH and OH methods produce nearly identical BVHs, with OH_6 being slightly faster, however also slower to construct. Its construction however is still much faster than the construction of either RDH or the blended method.

The RDH and blended methods produce the fastest BVHs for this scene. However as we have learned the RDH method is very sensitive to ray distributions that don't intersect or don't nearly miss a large chunk of triangles. So overall the best BVH seems to be produced by the blended method.

Metrics	naive	SAH	RDH_6_10	blended_6_4	OH_6
AoAI	18.33	6.82	6.59	6.50	6.80
AoMI	76.39	30.44	31.11	30.79	30.48
MoAI	32.32	11.83	10.25	10.12	11.55
MoMI	131	45	45	45	45
AoAT	73.79	69.41	54.58	55.10	70.37
AoMT	200.73	190.69	153.42	155.79	192.23
MoAT	111.43	106.23	69.95	70.88	106.00
MoMT	265	245	187	193	253
C	0	0.56	0.56	0.56	1.12
A	0	0	1766.68	2468.55	94.82
P	0	0	0	0	11.2

Table 10: Rendering and construction (in millions) metrics for LOA scene

In Table 10 we see the rendering and construction metrics for the BVH with

the best "speed" metric from each category for the LOA scene.

Observations from the previous scene also apply here. The RDH and blended BVHs are the best, however also the most costly to construct. The SAH and OH BVHs produce very similar results for all metrics, and the naive BVH is the worst performing in all rendering metrics.

Metrics	naive	SAH	RDH_10_12	blended_20_20	OH_20
AoAI	23.88	9.46	9.38	9.35	9.49
AoMI	59.62	27.24	114.95	28.87	26.10
MoAI	45.91	18.42	15.96	15.82	18.46
MoMI	95	41	464	41	41
AoAT	99.18	88.52	73.51	73.77	106.83
AoMT	188.82	174.81	165.27	172.27	200.31
MoAT	172.98	147.70	100.96	101.65	192.66
MoMT	273	237	211	215	309
C	0	0.56	0.35	0.56	1.15
A	0	0	2509.4	1803.02	414.49
P	0	0	0	0	53.58

Table 11: Rendering and construction (in millions) metrics for HOA scene

In Table 11 we see the rendering and construction metrics for the BVH with the best "speed" metric from each category for the HOA scene.

Here the slowest BVH is the naive one as per usual, however the second slowest one is the OH BVH. Since this is a modified version of the method described in (Vinkler et al., 2012), that was created to perform best in highly occluded scenes we can conclude that our modifications did not keep this quality of the BVH subdivision method, since this is the most occluded scene.

The SAH method is once again lagging behind both the RDH and blended method which perform very similarly again. However we can once again see indication of the RDH construction being sensitive to viewpoint placements from its high MoMI value.

Based on this investigation the fastest and most stable BVH subdivision method is the blended method. However its construction takes orders of magnitude more time than the construction of a BVH with a simple SAH method, which provides comparable performance in all circumstances.

References

Bittner, J., & Havran, V. (2009). Rdh: ray distribution heuristics for construction of spatial data structures. , 51–58. Retrieved from `https://doi.org/10.1145/1980462.1980475` doi: 10.1145/1980462.1980475

Meister, D., Ogaki, S., Benthin, C., Doyle, M. J., Guthe, M., & Bittner, J. (2021). A survey on bounding volume hierarchies for ray tracing. , *40*(2), 683–712.

Vinkler, M., Havran, V., & Sochor, J. (2012). Visibility driven bvh build up algorithm for ray tracing. *Computers Graphics*, *36*(4), 283-296. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0097849312000362` (Applications of Geometry Processing) doi: https://doi.org/10.1016/j.cag.2012.02.013